

Trabajo fin de grado

Desarrollo de un Tutor Socrático basado en LLMs para la enseñanza
de Programación



Ignacio García Hernanz

Escuela Politécnica Superior
Universidad Autónoma de Madrid
C/ Francisco Tomás y Valiente nº 11

**UNIVERSIDAD AUTÓNOMA DE MADRID
ESCUELA POLITÉCNICA SUPERIOR**



Grado en Ingeniería Informática

TRABAJO FIN DE GRADO

**Desarrollo de un Tutor Socrático basado en LLMs
para la enseñanza de Programación**

SocratiCode

**Autor: Ignacio García Hernanz
Tutor: Oscar Delgado Ben Mohatar**

abril 2026

Todos los derechos reservados.

Queda prohibida, salvo excepción prevista en la Ley, cualquier forma de reproducción, distribución comunicación pública y transformación de esta obra sin contar con la autorización de los titulares de la propiedad intelectual.

La infracción de los derechos mencionados puede ser constitutiva de delito contra la propiedad intelectual (*arts. 270 y sgts. del Código Penal*).

DERECHOS RESERVADOS

© Fecha de entrega por UNIVERSIDAD AUTÓNOMA DE MADRID
Francisco Tomás y Valiente, n.º 1
Madrid, 28049
Spain

Ignacio García Hernanz

Desarrollo de un Tutor Socrático basado en LLMs para la enseñanza de Programación

Ignacio García Hernanz

C\ Francisco Tomás y Valiente N.º 11

IMPRESO EN ESPAÑA – PRINTED IN SPAIN

Dedicatória...

Cita famosa...

Autor

PREFACIO

Este estilo de $\text{\LaTeX} 2_{\epsilon}$ ha sido diseñado con dos propósitos. El primer propósito es el de facilitar en lo posible la escritura de trabajos de fin de grado y de máster y de tesis doctorales. En ese sentido se han diseñado un conjunto de comandos que simplifican la escritura y diseño de estos trabajos pero que reducen en cierta forma las capacidades de los paquetes de $\text{\LaTeX}$ utilizados. Sin embargo, dado que los paquetes están incluidos en esta clase, pueden utilizarse directamente y hacer diseños más complejos pero si se hace esto se recomienda mantener una estética coherente con el resto del documento.

El segundo de los propósitos es que estos documentos mantengan una estética uniforme en la Universidad Autónoma de Madrid y fomentar una imagen corporativa en documentos tan relevantes como los trabajos de fin de grado o de máster y las tesis doctorales. Por ese motivo se recomienda mantener una coherencia estética en todo momento. El diseño facilita esa coherencia pero es posible salirse del diseño si se mantiene dicha coherencia.

Como creador de este estilo espero fervientemente que al usar este estilo te sientas cómodo y te facilite la escritura de un documento que es muy relevante en esta etapa de tu vida. Para facilitártela aún más, el código fuente de este documento también está disponible en tu ordenador o en overleaf para que te sirva a modo de ejemplo.

Eloy Anguiano Rey

AGRADECIMIENTOS

En primer lugar me gustaría agradecer a la Escuela Politécnica Superior por su apoyo para la creación de esta clase y que sea el formato básico para la creación de tesis, trabajos fin de grado y trabajos fin de master.

En particular quiero destacar el trabajo realizado por Fernando López-Colino por su apoyo en la comisión de imagen institucional y por sus comentarios para mejorar este estilo.

También quiero tener un recuerdo para Carmen Navarrete Navarrete dado que este estilo comencé a crearlo a partir de sus necesidades a la hora de escribir la tesis. Y por supuesto a no quiero olvidarme de mi esposa e hijos que han servido de conejillos de indias en sus correspondientes trabajos fin de master y de grado. No quiero olvidar a todos los estudiantes que me pidieron este estilo y lo han usado para presentar sus trabajos pero son muchos y podría olvidarme de alguno, por tanto, mi agradecimiento en general a todos ellos.

RESUMEN

En nuestra Escuela se producen un número considerable de documentos, tantos docentes como investigadores. Nuestros alumnos también contribuyen a esta producción a través de sus trabajos de fin de grado, máster y tesis. El objetivo de este material es facilitar la edición de todos estos documentos y a la vez fomentar nuestra imagen corporativa, facilitando la visibilidad y el reconocimiento de nuestro Centro.

En este sentido se ha intentado diseñar un estilo de $\text{\LaTeX} 2_{\epsilon}$ que mantenga una imagen corporativa y con comandos simples que permitan mantener la imagen corporativa con la calidad necesaria sin olvidar las necesidades del autor. Para ello se han creado un conjunto de comandos simples en torno a paquetes complejos. Estos comandos permiten realizar la mayoría de las operaciones que un documento de este tipo pueda necesitar.

Así mismo se puede controlar un poco el diseño del documento a través de las opciones del estilo pero siempre manteniendo la imagen institucional.

PALABRAS CLAVE

Diseño de documento, $\text{\LaTeX} 2_{\epsilon}$, thesis, trabajo fin de grado, trabajo fin de master

ABSTRACT

In our School a considerable number of documents are produced, as many aducational as research. Our students also contribute to this production through his final degree, master and thesis projects. The objective of this material is to facilitate the editing of all these documents and at the same time to promote our corporate image, facilitating the visibility and recognition of our center.

In this sense we have tried to design a style of $\text{\LaTeX}2_{\epsilon}$ that maintains a corporate image and with simple commands that allow to maintain the corporate image with the necessary quality without forgetting the needs of the author. For this, a set of simple commands have been created around complex packages. These commands allow you to perform most of the operations that a document of this type may need.

Likewise, you can control a little the design of the document through the options of the style but always maintaining the institutional image.

KEYWORDS

Document design, $\text{\LaTeX}2_{\epsilon}$, thesis, final degree project, final master project

ÍNDICE

1	Introducción	1
1.1	Motivación	1
1.2	Objetivos	2
1.3	Estructura del Proyecto	2
2	Estado del arte	5
2.1	Fundamentos Pedagógicos en la Enseñanza de la Programación	5
2.1.1	Dificultades en el aprendizaje inicial (CS1)	5
2.1.2	El Método Socrático como mitigación	6
2.2	Inteligencia Artificial Generativa en la Educación	6
2.2.1	Riesgos del facilismo algorítmico	6
2.2.2	Sistemas de tutoría guiada (Casos de Éxito)	7
2.2.3	Ingeniería de Prompts en educación	8
2.3	Modelos de Lenguaje Extenso (LLMs) y Despliegue Local	8
2.3.1	APIs en la nube vs. Inferencia Local	8
2.3.2	Optimización de modelos ligeros (Llama 3.2)	8
2.4	Seguridad y Aislamiento de Procesos (Sandboxing)	8
2.4.1	Vectores de amenaza en plataformas interactivas	8
2.4.2	Primitivas de aislamiento a nivel de Kernel	8
2.4.3	Motores de ejecución efímeros	8
2.5	Arquitecturas Web Educativas y Transmisión de Datos	8
2.5.1	Paradigma de Aplicación Web Desacoplada	8
2.5.2	Protocolos de comunicación en tiempo real	9
2.6	Conclusiones del Estado del Arte	9
2.6.1	Síntesis y propuesta de valor	9
3	Análisis y diseño	11
3.1	Metodología	11
3.2	Descripción del sistema - SocratiCode	11
3.3	Requisitos funcionales y no funcionales	11
3.3.1	Requisitos funcionales	11
3.3.2	Requisitos no funcionales	11
3.4	Casos de uso	11
3.5	Diagrama de modelo de datos (Clases/Entidad-Relación)	11

3.6	Diagrama de componentes y arquitectura	11
3.7	Conclusiones	11
4	Desarrollo e implementación	13
4.1	Entorno de programación	13
4.2	Tecnologías y librerías	13
4.2.1	Framework Backend (Django / Python)	13
4.2.2	Framework Frontend (Vue 3 / Vite)	13
4.2.3	Motor de Inteligencia Artificial (Ollama / Llama 3.2)	13
4.2.4	Motor de compilación y ejecución (Piston / Docker)	13
4.2.5	Control de versiones y despliegue (Git / GitHub)	13
4.3	Implementación	13
4.3.1	Arquitectura del Chat y Moderación de respuestas	13
4.3.2	Integración de ejecución de código en tiempo real	13
4.4	Interfaz	13
4.4.1	Navegación y vista principal del tutor	13
4.4.2	Personalización y editor de código	13
4.5	Conclusiones	14
5	Pruebas	15
5.1	Pruebas de seguridad e integridad (Pentesting del Sandbox)	15
5.2	Encuesta de experiencia de usuario y validación pedagógica	15
6	Conclusiones y trabajo futuro	17
6.1	Conclusiones	17
6.2	Trabajo futuro	17
	Bibliografía	19
	Apéndices	21
A	Sprints desarrollados	23
B	Diagramas y casos de uso	25
C	Navegación e interfaces extra	27
D	Resultados de encuesta y auditoría de seguridad	29

LISTAS

Lista de algoritmos

Lista de códigos

Lista de cuadros

Lista de ecuaciones

Lista de figuras

Lista de tablas

Lista de cuadros

INTRODUCCIÓN

El presente capítulo sirve como punto de partida y ofrece una panorámica global del Trabajo Fin de Grado. Su propósito es contextualizar el desarrollo de SocratiCode y establecer las bases sobre las que se fundamenta el proyecto.

El recorrido comienza en la Sección 1.1, donde se analizan las diferentes razones que han impulsado su realización. En ella se destaca la creciente necesidad de apoyar la enseñanza de la programación haciendo uso de capacidades basadas en Inteligencia Artificial y aplicando metodologías de aprendizaje activo como el diálogo socrático. A continuación, la Sección 1.2 define de manera concreta las metas a conseguir con este sistema, abarcando tanto las exigencias desde el punto de vista del desarrollo técnico como el impacto educativo que se espera lograr. Para concluir, la Sección 1.3 proporciona un breve mapa de lectura de la presente memoria, delineando la organización y la temática que el lector encontrará a lo largo de los capítulos venideros.

1.1. Motivación

En los últimos años, la Inteligencia Artificial ha evolucionado hasta alcanzar la capacidad de generar, testear y ejecutar código con una competencia equiparable a la de un ingeniero cualificado. No obstante, continúa siendo una herramienta que requiere de un usuario capaz de guiarla y supervisar sus resultados. Un estudiante que se inicia en la programación carece de dichas facultades analíticas, sin embargo, tiene a su disposición una tecnología capaz de resolver, en cuestión de segundos, los retos iniciales inherentes al aprendizaje del desarrollo de software.

La prohibición del uso de estas herramientas en el ámbito académico puede resultar contraproducente, puesto que el aprendizaje de la programación puede generar altos niveles de frustración en principiantes. Esta situación conlleva el riesgo de que el alumnado abandone el estudio o recurra a soluciones automatizadas que impidan el desarrollo del pensamiento computacional, facultad indispensable para establecer los fundamentos técnicos que posteriormente, permitirán aprovechar todo el potencial de estas tecnologías avanzadas de manera eficaz.

Ante este nuevo paradigma, el enfoque del programador profesional no debe residir exclusivamente

en la sintaxis de los diversos lenguajes de programación. Por el contrario, el valor diferencial se desplaza hacia el diseño de sistemas, la descomposición modular en subtarear para su implementación y la selección estratégica de las herramientas y tecnologías más adecuadas para cada requerimiento técnico.

Bajo este contexto, **SocratiCode** nace como una respuesta técnica y pedagógica a la actual dicotomía entre la prohibición del uso de la IA y la automatización excesiva en el aprendizaje. El proyecto propone una arquitectura basada en modelos de lenguaje locales que, en lugar de generar soluciones directas, actúan como tutores socráticos. Al integrar un entorno de ejecución seguro con una asistencia iterativa, la plataforma busca transformar la IA en un catalizador del pensamiento computacional, garantizando que el alumno mantenga el control del flujo lógico y el esfuerzo cognitivo en su aprendizaje.

1.2. Objetivos

El objetivo principal de este Trabajo Fin de Grado es diseñar, implementar y validar una plataforma educativa de programación que integre un tutor basado en inteligencia artificial local con una metodología socrática de enseñanza y un entorno de ejecución de código aislado y seguro. Para alcanzar dicho objetivo, se plantean los siguientes objetivos específicos:

- Investigar el estado del arte en sistemas de tutoría inteligente y plataformas de aprendizaje de programación, analizando las capacidades actuales de la IA generativa para fomentar el pensamiento computacional.
- Evaluar y seleccionar las soluciones tecnológicas más eficientes para la inferencia local de modelos de lenguaje (Ollama) y la orquestación de procesos aislados, garantizando un compromiso óptimo entre privacidad, rendimiento y seguridad.
- Diseñar una arquitectura de sistema integral que armonice el procesamiento de lenguaje natural, un motor de ejecución de código efímero y una interfaz de usuario reactiva orientada a la experiencia pedagógica.
- Adoptar una metodología de desarrollo iterativa basada en principios ágiles, permitiendo la evolución progresiva del sistema desde un Producto Mínimo Viable (MVP) hasta una plataforma robusta mediante ciclos de refinamiento continuo.
- Elaborar la documentación técnica detallada siguiendo estándares de ingeniería del software, abarcando desde la captura de requisitos y casos de uso hasta el modelado de la arquitectura y diagramas estructurales.
- Validar la plataforma mediante auditorías de seguridad sobre el entorno de ejecución y la realización de cuestionarios de experiencia de usuario para contrastar la efectividad real del tutor socrático.

1.3. Estructura del Proyecto

La presente memoria se organiza en seis capítulos cuyo contenido se describe a continuación:

El Capítulo 1 introduce el contexto y la motivación que justifican la realización del proyecto, expone

los principales objetivos técnicos y pedagógicos planteados, y proporciona una visión general de la estructura del documento.

El Capítulo 2 recoge el estado del arte, abordando los fundamentos pedagógicos del método socrático, el impacto de la Inteligencia Artificial generativa en la educación y las ventajas de los modelos de lenguaje de ejecución local. Asimismo, se analizan las tecnologías de aislamiento de procesos (sandboxing) y las arquitecturas web necesarias para sistemas educativos interactivos en tiempo real.

El Capítulo 3 presenta el análisis y diseño del sistema, describiendo la metodología de trabajo seguida, los requisitos funcionales y no funcionales, los casos de uso, el modelo de datos y la arquitectura de componentes.

El Capítulo 4 detalla el proceso de desarrollo e implementación, describiendo el entorno de trabajo, las tecnologías empleadas y las decisiones de diseño más relevantes adoptadas durante la construcción del sistema.

El Capítulo 5 recoge las pruebas realizadas sobre el sistema, tanto las pruebas de seguridad del sandbox como la validación pedagógica con usuarios finales.

El Capítulo 6 expone las conclusiones del proyecto y propone líneas de trabajo futuro.

ESTADO DEL ARTE

En este capítulo se analiza el contexto técnico y educativo en el que se enmarca **SocratiCode**. En primer lugar, se examinan los fundamentos pedagógicos que justifican la necesidad de nuevas herramientas en la enseñanza de la informática. Posteriormente, se realiza un recorrido por las soluciones tecnológicas actuales y los casos de éxito que han integrado la Inteligencia Artificial en el aula.

2.1. Fundamentos Pedagógicos en la Enseñanza de la Programación

Para entender el propósito de **SocratiCode**, es necesario analizar primero la realidad de las aulas de informática. El desarrollo de una plataforma de este tipo no responde solo a un interés tecnológico, sino a la búsqueda de una solución para dos problemas críticos: las barreras cognitivas que enfrentan los estudiantes principiantes y la imposibilidad de ofrecer una tutoría personalizada en entornos masificados. A continuación, se detallan estos desafíos y la propuesta del método socrático como vía para superarlos.

2.1.1. Dificultades en el aprendizaje inicial (CS1)

La enseñanza de la programación en los cursos introductorios, comúnmente denominados CS1 (*Computer Science 1*), representa uno de los desafíos pedagógicos más críticos en la educación superior contemporánea [1]. Históricamente, los programas académicos de informática experimentan altas tasas de abandono en estas etapas iniciales, con porcentajes de deserción que frecuentemente oscilan entre el 30 % y el 40 %. Esta problemática no responde únicamente a una falta de interés, sino a la complejidad de asimilar simultáneamente tanto las reglas gramaticales del código como la lógica necesaria para resolver problemas.

Para un estudiante que se enfrenta por primera vez a la programación, el entorno de desarrollo puede resultar abrumador. La combinación de conceptos abstractos y la rigidez de las herramientas genera una frustración profunda. Al no contar todavía con un modelo mental sólido, los alumnos tienen

grandes dificultades para entender por qué su código no funciona, sintiéndose a menudo incapaces de avanzar de forma autónoma. Idealmente, un profesor debería estar disponible para ofrecer pistas en el momento justo, pero en las facultades actuales, con clases masificadas, es físicamente imposible atender a cada estudiante de forma individualizada. Esta limitación es la que hace necesario buscar sistemas de tutoría automatizados que ofrezcan ese apoyo constante en tiempo real.

2.1.2. El Método Socrático como mitigación

Para resolver esta falta de acompañamiento sin caer en el extremo de resolverle el trabajo al alumno, la pedagogía moderna propone estrategias de instrucción guiada, destacando entre ellas el método socrático [2]. Este enfoque no busca dar respuestas, sino establecer un diálogo donde el tutor guía al estudiante mediante preguntas pequeñas y estratégicas. El objetivo es que el alumno "tire del hilo" de su propio razonamiento hasta descubrir, por sí mismo, dónde está el fallo en su lógica [3]. Al obligar al estudiante a verbalizar y reflexionar sobre lo que está intentando hacer, se consigue un aprendizaje mucho más profundo que el que se obtiene con una corrección automática [4].

Este apoyo, conocido como *scaffolding*, permite que el alumno no se rinda ante el primer bloqueo, ya que la IA actúa como un soporte temporal que se adapta a su nivel [5]. En el contexto actual de la Inteligencia Artificial, este método se vuelve una pieza fundamental para garantizar que la tecnología sea una ayuda y no una herramienta de plagio. Mediante una cuidada ingeniería de *prompts*, podemos configurar sistemas que emulen la paciencia de un tutor humano, combatiendo así el riesgo de que el alumno delegue su esfuerzo cognitivo en la máquina y pierda la oportunidad de aprender realmente a programar [6].

2.2. Inteligencia Artificial Generativa en la Educación

La llegada de la IA generativa ha cambiado por completo la forma de enseñar. En programación, la capacidad de los modelos de lenguaje (LLM) para entender y corregir código es una oportunidad enorme para personalizar el aprendizaje. Sin embargo, esto también trae riesgos: si la herramienta le da la solución hecha al alumno, este puede perder la capacidad de razonar por sí mismo. El reto ahora es diseñar herramientas que ayuden a aprender sin convertirse en un atajo que evite el esfuerzo de pensar.

2.2.1. Riesgos del facilismo algorítmico

El uso masivo de herramientas como ChatGPT o GitHub Copilot ha hecho que conseguir la solución a cualquier problema de programación sea instantáneo. Cuando estas tecnologías se usan para

aprender sin ningún tipo de límite pedagógico, aparece lo que llamamos "facilismo algorítmico". Los estudiantes suelen dejar que la IA resuelva todo el problema y se limitan a copiar y pegar el código en sus proyectos [6]. Este atajo evita el esfuerzo mental que es realmente necesario para entender cómo programar. Estudios recientes muestran que esta dependencia no solo frena el pensamiento crítico, sino que crea una "ilusión de competencia": el alumno tiene un código que funciona, pero no entiende por qué [7]. A la larga, esto se traduce en un peor rendimiento y en una pérdida de aprendizaje que se hace evidente al llegar a cursos más difíciles.

2.2.2. Sistemas de tutoría guiada (Casos de Éxito)

Para evitar estos problemas sin renunciar a la IA, algunas instituciones han diseñado sistemas que siguen reglas pedagógicas muy estrictas. El referente más claro es el CS50 Duck de la Universidad de Harvard. Este asistente está integrado directamente en CS50.dev, una versión de Visual Studio Code adaptada para la nube mediante GitHub Codespaces. No funciona como un simple chat abierto, sino como un sistema complejo que utiliza una arquitectura RAG para conectar las dudas del alumno con el temario oficial del curso y filtros de seguridad que bloquean intentos de inyección de prompts. Toda su lógica operativa se gestiona desde un backend centralizado que utiliza archivos de configuración YAML para ajustar dinámicamente el comportamiento de GPT-4, asegurando que el modelo nunca entregue la solución directa, sino pistas y preguntas de seguimiento que fomenten el razonamiento independiente y la técnica del rubberducking [8].

Para gestionar los costes de la API comercial y potenciar el aprendizaje profundo, Harvard implementó un ingenioso sistema de "corazones" que limita las consultas de los estudiantes, obligándoles a reflexionar antes de formular sus dudas para no agotar sus recursos. Aunque este enfoque es pedagógicamente brillante, su dependencia de modelos en la nube genera gastos operativos elevados y plantea retos sobre la privacidad al enviar datos a servidores externos [9].

En una línea parecida el asistente CodeAid ha validado un modelo de ayuda estructurada en clases masivas. A diferencia de un chat convencional este software ofrece un menú de herramientas específicas para el alumno como la explicación de conceptos técnicos o la resolución de dudas teóricas. Para la revisión del código el sistema habilita una caja de texto donde el usuario puede pegar sus líneas pero sin capacidad de ejecución directa. A partir de ahí el programa sigue un proceso interno basado en una técnica de *few-shot prompting*, donde se entrena al modelo con ejemplos previos de interacciones pedagógicas para asegurar que el tono sea siempre de apoyo. Bajo esta arquitectura la IA primero genera la solución correcta de forma invisible para garantizar la precisión pero después la traduce en un pseudocódigo que orienta al estudiante sin darle nunca la respuesta final. Este método tiene el beneficio de forzar al usuario a que reflexione sobre la lógica del ejercicio y tenga que escribir él mismo cada línea en su editor lo que asegura que el proceso de aprendizaje sea efectivo y se evite el simple copiado de soluciones. Además esta plataforma también anonimiza los datos personales al

igual que hace el sistema de Harvard aunque no cuenta con una protección tan estricta contra los ataques de inyección de instrucciones [10].

2.2.3. Ingeniería de Prompts en educación

Lograr que un modelo de lenguaje convencional funcione como un tutor socrático que sepa limitarse depende de un diseño de prompts muy cuidado. No basta con darle un papel a la IA sino que hay que estructurar sus instrucciones para que use métodos como la dialéctica o la mayéutica [11]. El principal obstáculo para conseguirlo es que los modelos actuales están diseñados por defecto para actuar como asistentes que ofrecen respuestas directas y rápidas a cualquier consulta. Esta naturaleza choca con el entorno educativo donde el objetivo es que el sistema actúe como una guía que no regale el código final. Para superar esta limitación las investigaciones proponen una evaluación jerárquica donde la IA primero identifica si la pregunta del alumno es general o específica y analiza el estado de su código antes de responder. Al aplicar esta lógica el modelo puede formular preguntas que parten del nivel de conocimiento real del estudiante para que sea él quien descubra la solución mediante su propio razonamiento [2].

2.3. Modelos de Lenguaje Extenso (LLMs) y Despliegue Local

2.3.1. APIs en la nube vs. Inferencia Local

2.3.2. Optimización de modelos ligeros (Llama 3.2)

2.4. Seguridad y Aislamiento de Procesos (Sandboxing)

2.4.1. Vectores de amenaza en plataformas interactivas

2.4.2. Primitivas de aislamiento a nivel de Kernel

2.4.3. Motores de ejecución efímeros

2.5. Arquitecturas Web Educativas y Transmisión de Datos

2.5.1. Paradigma de Aplicación Web Desacoplada

2.5.2. Protocolos de comunicación en tiempo real

2.6. Conclusiones del Estado del Arte

2.6.1. Síntesis y propuesta de valor

ANÁLISIS Y DISEÑO

3.1. Metodología

3.2. Descripción del sistema - SocratiCode

3.3. Requisitos funcionales y no funcionales

3.3.1. Requisitos funcionales

3.3.2. Requisitos no funcionales

3.4. Casos de uso

3.5. Diagrama de modelo de datos (Clases/Entidad-Relación)

3.6. Diagrama de componentes y arquitectura

3.7. Conclusiones

DESARROLLO E IMPLEMENTACIÓN

4.1. Entorno de programación

4.2. Tecnologías y librerías

4.2.1. Framework Backend (Django / Python)

4.2.2. Framework Frontend (Vue 3 / Vite)

4.2.3. Motor de Inteligencia Artificial (Ollama / Llama 3.2)

4.2.4. Motor de compilación y ejecución (Piston / Docker)

4.2.5. Control de versiones y despliegue (Git / GitHub)

4.3. Implementación

4.3.1. Arquitectura del Chat y Moderación de respuestas

4.3.2. Integración de ejecución de código en tiempo real

4.4. Interfaz

4.4.1. Navegación y vista principal del tutor

4.4.2. Personalización y editor de código

4.5. Conclusiones

PRUEBAS

- 5.1. Pruebas de seguridad e integridad (Pentesting del Sandbox)
- 5.2. Encuesta de experiencia de usuario y validación pedagógica

CONCLUSIONES Y TRABAJO FUTURO

6.1. Conclusiones

6.2. Trabajo futuro

BIBLIOGRAFÍA

- [1] Z. Alshaikh, L. Tamang, and V. Rus, “A socratic tutor for source code comprehension,” in *International Conference on Artificial Intelligence in Education (AIED)*, pp. 15–19, Springer, 2020.
- [2] P. Kargupta, I. Agarwal, D. H. Tur, and J. Han, “Instruct, not assist: Llm-based multi-turn planning and hierarchical questioning for socratic code debugging,” in *Findings of the Association for Computational Linguistics: EMNLP 2024*, pp. 9443–9458, 2024.
- [3] E. Al-Hossami, R. Bunescu, J. Smith, and R. Teehan, “Can language models employ the socratic method? experiments with code debugging,” in *Proceedings of the 55th ACM Technical Symposium on Computer Science Education V. 1*, pp. 53–59, 2024.
- [4] V. Aleven and K. R. Koedinger, “An effective metacognitive strategy: Learning by doing and explaining with a computer-based cognitive tutor,” *Cognitive Science*, vol. 26, no. 2, pp. 147–179, 2002.
- [5] D. Wood, J. S. Bruner, and G. Ross, “The role of tutoring in problem solving,” *Journal of Child Psychology and Psychiatry*, vol. 17, no. 2, pp. 89–100, 1976.
- [6] B. Chen, C. M. Lewis, M. West, and C. Zilles, “Plagiarism in the age of generative ai: Cheating method change and learning loss in an intro to cs course,” in *Proceedings of the Eleventh ACM Conference on Learning at Scale*, pp. 75–85, 2024.
- [7] J. Prather *et al.*, “Widening the gap: Genai tools can compound the metacognitive difficulties novices face when learning to program,” in *Proceedings of the 2024 on Innovation and Technology in Computer Science Education V. 1*, pp. 276–282, 2024.
- [8] D. J. Malan, “Cs50x 2026 - artificial intelligence.” Vídeo en YouTube, 2026. Accedido el 24 de abril de 2026.
- [9] R. Liu, C. Zenke, C. Liu, and D. J. Malan, “Teaching cs50 with ai: Leveraging generative artificial intelligence in computer science education,” in *Proceedings of the 55th ACM Technical Symposium on Computer Science Education V. 1*, 2024.
- [10] M. Kazemitabaar, R. Ye, X. Wang, A. Z. Henley, P. Denny, M. Craig, and T. Grossman, “Codeaid: Evaluating a classroom deployment of an llm-based programming assistant that balances student and educator needs,” in *Proceedings of the CHI Conference on Human Factors in Computing Systems*, pp. 1–17, 2024.
- [11] E. Y. Chang, “Prompting large language models with the socratic method,” in *IEEE Computing and Communication Workshop and Conference (CCWC)*, 2023.

APÉNDICES

SPRINTS DESARROLLADOS

DIAGRAMAS Y CASOS DE USO

NAVEGACIÓN E INTERFACES EXTRA



RESULTADOS DE ENCUESTA Y AUDITORÍA DE SEGURIDAD



Universidad Autónoma
de Madrid